

Special Need Vehicle Rental

Application Architecture & Developer Guide

Version: 1.0.0 **Last Updated:** June 2026 **Jurisdiction:** New South Wales, Australia **Audience:** Software developers, DevOps engineers, technical leads

Table of Contents

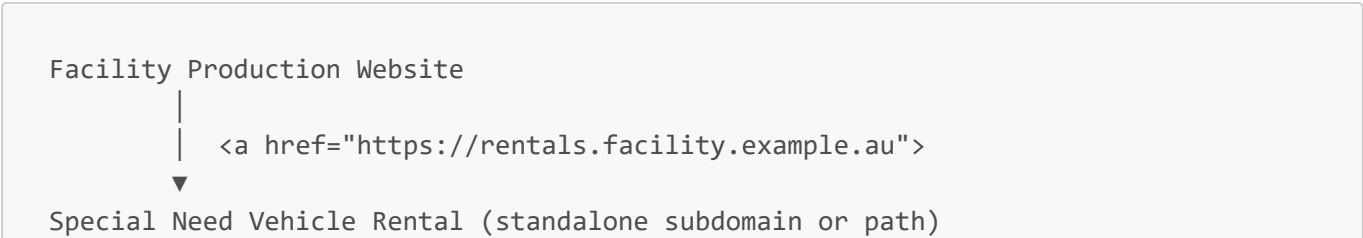
- 1. Overview & Purpose
- 2. System Architecture
- 3. Technology Stack
- 4. Data Model
- 5. API Specification
- 6. Frontend Architecture
- 7. Payment Form (MVP)
- 8. Security & Compliance
- 9. Developer Setup
- 10. Database Migrations & Seeding
- 11. Testing Strategy
- 12. Deployment Guide
- 13. Cloud Hosting Cost Estimate
- 14. Monitoring & Observability
- 15. Backup & Recovery
- 16. Handoff Deliverables

1. Overview & Purpose

Special Need Vehicle Rental is a lean, secure web application that allows the public to browse and book the facility's **already-owned vehicles** during their idle hours — when they are not in use for resident transport. It is presented as a single external link embedded in the existing facility production website. The application is designed to be:

- **Minimal in scope** — up to 100 vehicles, low traffic, single region
- **Low-cost** — targets under AUD \$35/month total infrastructure
- **NSW-compliant** — privacy, accessibility, GST and insurance requirements observed
- **Extensible** — payment gateway integration is deferred to Phase 2; all other scaffolding is in place

Integration Pattern

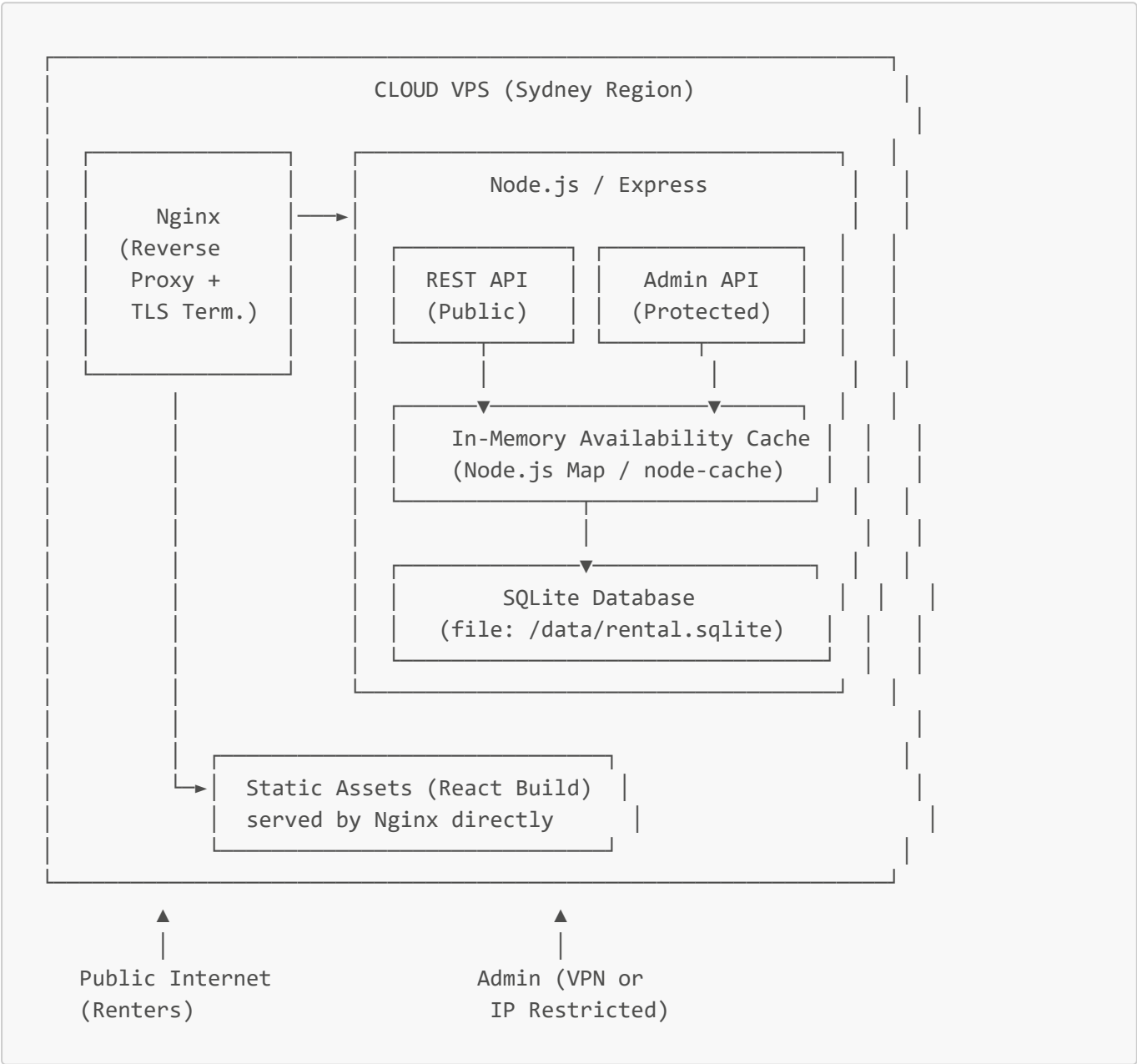




The application lives on its own subdomain (e.g., `rentals.facilitydomain.com.au`) and shares no session or data with the parent site. Cross-origin resource sharing (CORS) is restricted to the parent domain.

2. System Architecture

2.1 High-Level Component Diagram



2.2 Architectural Decisions

Decision	Choice	Rationale
Runtime	Node.js 20 LTS	Wide ecosystem, async I/O suits booking concurrency

Decision	Choice	Rationale
Database	SQLite	Zero managed-DB cost; 100 vehicles = < 5 MB data
Caching	In-process Map / node-cache	Availability queries are hot; TTL = 30 seconds
Frontend	React 18 SPA	Modern component model; CRA or Vite for build
Reverse Proxy	Nginx	TLS termination, gzip, static file serving
Process Manager	PM2	Auto-restart, cluster mode, log management
TLS	Let's Encrypt / Certbot	Free, auto-renewing

2.3 Timezone & Locale

- All dates stored in SQLite as **ISO 8601 UTC** strings (`2026-06-15T04:00:00Z`)
- All dates displayed to users in **Australia/Sydney** (AEST UTC+10 / AEDT UTC+11)
- The UI explicitly labels timezone on every date/time field
- Currency: **AUD** with two decimal places; all prices include GST (10%) where applicable

3. Technology Stack

3.1 Backend

Layer	Technology	Version
Runtime	Node.js	20.x LTS
Framework	Express	4.x
Database driver	better-sqlite3	9.x
Caching	node-cache	5.x
Validation	zod	3.x
Auth (admin)	express-session + bcrypt	—
Rate limiting	express-rate-limit	7.x
Email	nodemailer (SMTP)	6.x
Logging	pino	8.x
Process mgr	PM2	5.x

3.2 Frontend

Layer	Technology	Version
Framework	React	18.x
Build tool	Vite	5.x

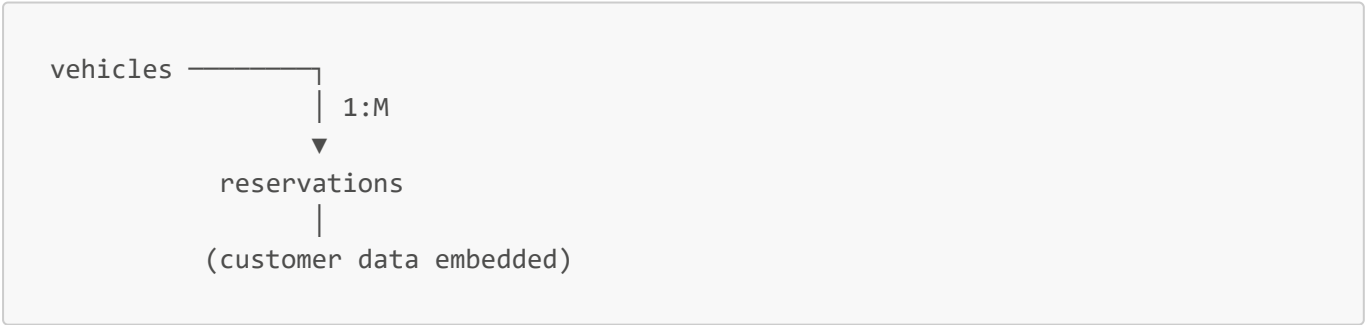
Layer	Technology	Version
Routing	React Router	6.x
Date/Time	date-fns-tz	3.x
UI library	shadcn/ui + Tailwind CSS	—
Forms	react-hook-form + zod	—
Availability picker	react-datepicker	6.x
Accessibility	aria-compliant components (see §8.4)	—

3.3 Infrastructure

Component	Technology
VPS	DigitalOcean / AWS Lightsail / EC2 t4g.nano
Web server	Nginx 1.24+
TLS	Let's Encrypt (Certbot)
OS	Ubuntu 22.04 LTS
Backups	Cron → rsync to S3-compatible storage
CI/CD (optional)	GitHub Actions → SSH deploy

4. Data Model

4.1 Entity-Relationship Overview



4.2 Full SQL Schema

```
-- =====
-- migrations/001_initial_schema.sql
-- =====
PRAGMA journal_mode = WAL;           -- concurrent readers while writing
PRAGMA foreign_keys = ON;

-- Vehicle catalogue
CREATE TABLE IF NOT EXISTS vehicles (
```

```

    id                INTEGER PRIMARY KEY AUTOINCREMENT,
    name              TEXT    NOT NULL,
    type              TEXT    NOT NULL, -- 'sedan' | 'wagon' | 'van' |
'wheelchair'
    capacity          INTEGER NOT NULL DEFAULT 1,
    plate             TEXT    UNIQUE,
    accessibility_notes TEXT,          -- ramp, wheelchair lift, hand controls,
etc.
    photos_json       TEXT,          -- JSON array of relative image paths
    hourly_rate_cents INTEGER NOT NULL DEFAULT 0,
    daily_rate_cents  INTEGER NOT NULL DEFAULT 0,
    buffer_minutes    INTEGER NOT NULL DEFAULT 30, -- cleaning gap between
bookings
    status            TEXT    NOT NULL DEFAULT 'active',
-- 'active' | 'maintenance' | 'retired'
    maintenance_until TEXT,          -- ISO UTC; set when status =
'maintenance'
    created_at        TEXT    NOT NULL DEFAULT (datetime('now')),
    updated_at        TEXT
);

-- Customer reservations
CREATE TABLE IF NOT EXISTS reservations (
    id                INTEGER PRIMARY KEY AUTOINCREMENT,
    vehicle_id        INTEGER NOT NULL REFERENCES vehicles(id),
    customer_name     TEXT    NOT NULL,
    customer_email    TEXT    NOT NULL,
    customer_phone    TEXT    NOT NULL,
    intended_use      TEXT,          -- free-text: 'medical appointment', etc.
    addons_json       TEXT,          -- JSON array of selected add-ons
    start_utc         TEXT    NOT NULL, -- ISO 8601 UTC
    end_utc           TEXT    NOT NULL,
    status            TEXT    NOT NULL DEFAULT 'pending',
-- 'pending' | 'confirmed' | 'picked_up' | 'completed'
-- | 'cancelled'
    price_cents       INTEGER NOT NULL DEFAULT 0,
    gst_cents         INTEGER NOT NULL DEFAULT 0,
    deposit_cents     INTEGER NOT NULL DEFAULT 0,
    payment_status    TEXT    NOT NULL DEFAULT 'none',
-- 'none' | 'form_captured' | 'paid' | 'refunded'
    payment_token     TEXT,          -- placeholder; replaced by gateway token
in Phase 2
    terms_accepted    INTEGER NOT NULL DEFAULT 0, -- boolean 0|1
    terms_accepted_at TEXT,          -- ISO UTC timestamp
    cancellation_reason TEXT,
    created_at        TEXT    NOT NULL DEFAULT (datetime('now')),
    updated_at        TEXT
);

-- Audit log (append-only)
CREATE TABLE IF NOT EXISTS audit_log (
    id                INTEGER PRIMARY KEY AUTOINCREMENT,
    entity            TEXT NOT NULL,          -- 'reservation' | 'vehicle'
    entity_id         INTEGER NOT NULL,

```

```

    action      TEXT NOT NULL,          -- 'create' | 'update' | 'cancel' |
    'complete'
    actor       TEXT,                  -- 'customer' | 'admin'
    detail_json TEXT,
    created_at  TEXT NOT NULL DEFAULT (datetime('now'))
);

-- Performance indexes
CREATE INDEX IF NOT EXISTS idx_res_vehicle_time
  ON reservations(vehicle_id, start_utc, end_utc);

CREATE INDEX IF NOT EXISTS idx_res_status
  ON reservations(status);

CREATE INDEX IF NOT EXISTS idx_res_email
  ON reservations(customer_email);

```

4.3 Availability Overlap Logic

A vehicle is **available** for interval [NEW_START, NEW_END) when no existing reservation satisfies:

```

-- Overlap condition (existing record conflicts with proposed interval)
SELECT id FROM reservations
WHERE vehicle_id = ?
  AND status IN ('pending', 'confirmed', 'picked_up')
  AND NOT (end_utc <= ? OR start_utc >= ?)
--                ↑NEW_START      ↑NEW_END

```

This uses the standard Allen interval complement: two intervals do **not** overlap if one ends before the other starts or vice versa. The NOT of that is the overlap. Implement inside a **BEGIN IMMEDIATE TRANSACTION** to prevent concurrent double-bookings.

5. API Specification

All endpoints prefix: `/api/v1` Content-Type: `application/json` Dates: ISO 8601 UTC strings in request/response bodies Authentication: Admin endpoints require session cookie; public endpoints are unauthenticated but rate-limited.

5.1 Public Endpoints

GET /api/v1/vehicles

Returns all active vehicles with pricing.

Response 200:

```
{
  "data": [
    {
      "id": 1,
      "name": "Toyota HiAce (Wheelchair)",
      "type": "van",
      "capacity": 4,
      "accessibility_notes": "Rear wheelchair ramp, 2 anchor points",
      "hourly_rate_aud": "20.00",
      "daily_rate_aud": "120.00",
      "buffer_minutes": 30,
      "photos": ["/uploads/vehicles/1/front.jpg"]
    }
  ]
}
```

GET /api/v1/availability

Returns vehicles available for a given time window.

Query parameters:

Param	Type	Required	Description
start	ISO 8601 UTC	Yes	Requested start time
end	ISO 8601 UTC	Yes	Requested end time
type	string	No	Filter by vehicle type

Response 200:

```
{
  "requested_start": "2026-06-15T04:00:00Z",
  "requested_end": "2026-06-16T04:00:00Z",
  "timezone_display": "Australia/Sydney",
  "available": [ { "id": 1, "name": "Toyota HiAce (Wheelchair)", ... } ],
  "unavailable": [ { "id": 2, "name": "Ford Territory", "reason": "booked" } ]
}
```

Caching: Results cached in-memory for 30 seconds per `start+end+type` key. Cache is invalidated on any reservation write.

POST /api/v1/reservations

Creates a new reservation. Validates availability atomically.

Request body:

```
{
  "vehicle_id": 1,
  "customer_name": "Jane Smith",
  "customer_email": "jane.smith@example.com",
  "customer_phone": "+61412345678",
  "intended_use": "Medical appointment",
  "start_utc": "2026-06-15T04:00:00Z",
  "end_utc": "2026-06-16T04:00:00Z",
  "terms_accepted": true
}
```

Responses:

Code	Meaning
201	Reservation created; returns full reservation object
409	Vehicle no longer available for requested window
422	Validation error (missing fields, invalid dates)
429	Rate limit exceeded

GET /api/v1/reservations/:id

Returns a reservation by ID. Requires customer email match (query param `email=`) for security.

PATCH /api/v1/reservations/:id/cancel

Customer self-cancellation within the policy window.

Request body:

```
{ "customer_email": "jane.smith@example.com", "reason": "Plans changed" }
```

POST /api/v1/payments/form

Captures payment form fields for an existing reservation. **Phase 1 only — no gateway processing.**

Request body:

```
{
  "reservation_id": 42,
  "cardholder_name": "Jane Smith",

```



```
"billing_email": "jane.smith@example.com",
"billing_phone": "+61412345678",
"card_last4": "4242",
"expiry_month": "12",
"expiry_year": "2028",
"amount_aud": "120.00"
}
```

Security: Raw card numbers are NEVER accepted or stored. The frontend must not transmit full card numbers to this endpoint. In Phase 1, the UI instructs users that payment will be confirmed by staff prior to pickup. Phase 2 replaces this with a Stripe Payment Element or equivalent, handling card data exclusively on the gateway's PCI-compliant servers.

5.2 Admin Endpoints (Protected)

All admin routes require a valid admin session obtained via `POST /api/v1/admin/login`.

Method	Path	Description
POST	<code>/admin/login</code>	Authenticate admin (username + password)
POST	<code>/admin/logout</code>	Invalidate session
GET	<code>/admin/reservations</code>	List all reservations (filterable)
PATCH	<code>/admin/reservations/:id</code>	Update status, add notes
GET	<code>/admin/vehicles</code>	List all vehicles including retired
POST	<code>/admin/vehicles</code>	Add new vehicle
PATCH	<code>/admin/vehicles/:id</code>	Update vehicle details / set maintenance
DELETE	<code>/admin/vehicles/:id</code>	Retire vehicle (soft-delete)
GET	<code>/admin/reports/csv</code>	Export reservations CSV
GET	<code>/admin/audit</code>	View audit log

6. Frontend Architecture

6.1 Page Structure

```
/                Landing page + vehicle browse
/availability    Availability search results
/vehicles/:id    Vehicle detail + booking initiation
/booking        Multi-step booking form
  Step 1: Personal details + intended use
  Step 2: Terms & conditions acceptance
  Step 3: Payment form (card capture placeholder)
/booking/confirmation Confirmation + reservation summary
```

```
/cancel/:id      Customer cancellation page
/admin/*         Admin UI (protected)
  /admin/login
  /admin/dashboard
  /admin/reservations
  /admin/vehicles
  /admin/reports
```

6.2 Availability Calendar Component

The availability picker should be a date-range selector that:

- Defaults to **Australia/Sydney** timezone
- Disables past dates
- Highlights dates where some vehicles are available (green) vs. fully booked (red/grey)
- Shows minimum rental duration (1 hour) and maximum (configurable, default 7 days)
- Displays buffer times visually so renters do not select a slot too close to an existing booking

6.3 Accessibility Requirements (WCAG 2.1 AA)

The application must conform to **WCAG 2.1 Level AA** as mandated by the Australian Government's Digital Service Standard and consistent with the *Disability Discrimination Act 1992 (Cth)*:

- All form inputs have associated `<label>` elements
- All interactive components are keyboard navigable (Tab/Shift+Tab, Enter, Space, Escape)
- Colour contrast ratio: minimum 4.5:1 for normal text, 3:1 for large text
- No information conveyed by colour alone (use icons + text)
- All images have descriptive `alt` text
- Error messages are programmatically associated with fields via `aria-describedby`
- Focus management: on step change, focus moves to the new step heading
- Date pickers include keyboard alternatives (manual date entry fields)
- ARIA live regions announce dynamic updates (e.g., "Vehicle no longer available")
- Use semantic HTML5 landmarks: `<header>`, `<main>`, `<nav>`, `<footer>`, `<section>`

7. Payment Form (MVP)

7.1 Fields

Field	UI Label	Stored?	Notes
<code>cardholder_name</code>	Name on card	Yes	Plain text
<code>card_last4</code>	Card number (last 4)	Yes	UI masks; never full number
<code>expiry_month</code>	Expiry month	Yes	MM format
<code>expiry_year</code>	Expiry year	Yes	YYYY format
<code>billing_email</code>	Email	Yes	Matches reservation

Field	UI Label	Stored?	Notes
<code>billing_phone</code>	Phone	Yes	Australian format
<code>amount_aud</code>	Amount (AUD, incl. GST)	Yes	Read-only, prefilled
<code>reservation_id</code>	—	Yes	Hidden field

7.2 Behaviour

1. Backend stores a **placeholder payment token** and sets `payment_status = 'form_captured'`
2. Admin is notified to manually confirm payment by phone or bank transfer before vehicle handover
3. Confirmation page states clearly: *"Your booking is held pending payment confirmation. Our staff will contact you within 2 business hours to finalise payment."*

7.3 Phase 2 Gateway Upgrade Path

Replace the form endpoint with a **Stripe Payment Element** (or Tyro / eWAY / Braintree for Australian-first options). The `payment_token` column stores the Stripe `PaymentIntent.id`. No card data ever passes through the application server in Phase 2.

8. Security & Compliance

8.1 Transport Security

- All traffic HTTPS; HTTP redirects to HTTPS via Nginx
- TLS 1.2 minimum; TLS 1.3 preferred
- HSTS header: `max-age=31536000; includeSubDomains`
- Let's Encrypt cert auto-renewing via Certbot systemd timer

8.2 Application Security

Control	Implementation
Input validation	Zod schemas on all API inputs (server-side) + client-side mirroring
SQL injection	Parameterised queries via better-sqlite3 (no string concatenation)
XSS	React's default JSX escaping; CSP header via Nginx
CSRF	SameSite=Strict cookies for admin session; public API is stateless
Rate limiting	100 req/15min for booking endpoints; 20/min for payment form
Admin auth	Bcrypt password hashing; session expiry 8 hours; optional IP allowlist
Secrets	Environment variables only; never in source code or SQLite
Logging	Pino structured logs; no PII in log bodies; separate audit table

8.3 Content Security Policy (Nginx)

```
add_header Content-Security-Policy
"default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline';
img-src 'self' data;; connect-src 'self'; frame-ancestors 'none';"
always;
```

8.4 Australian Privacy Principles (APPs) — Privacy Act 1988 (Cth)

APP	Obligation	Implementation
APP 1	Privacy policy published	/privacy page with data handling disclosure
APP 3	Collect only what is necessary	Collect only: name, email, phone, intended use
APP 5	Notify at collection	Inline notice on booking form
APP 6	Use only for stated purpose	Data used only for reservation management
APP 11	Secure personal information	HTTPS + access controls + limited retention
APP 12	Access to own information	Contact admin email stated on confirmation page

8.5 PCI DSS Considerations

- Phase 1 (form-only): avoid raw card data in transit; store only masked/last-4; document that full card entry is to be replaced
- Phase 2 (gateway): use a PCI-validated gateway's hosted fields / Payment Element — application server is out of PCI scope
- Never log card numbers at any phase

9. Developer Setup

9.1 Prerequisites

Tool	Version
Node.js	20.x LTS
npm	10.x
Git	2.40+
SQLite CLI	3.x (optional, for inspection)

9.2 Clone & Configure

```
git clone https://github.com/your-org/special-need-vehicle-rental.git car_rental
cd car_rental
cp .env.example .env
```

9.3 Environment Variables

```
# Server
PORT=8080
NODE_ENV=development
BASE_URL=http://localhost:8080

# Database
DATABASE_PATH=./data/rental.sqlite

# Admin credentials (change before deployment)
ADMIN_USERNAME=admin
ADMIN_PASSWORD_HASH=$2b$12$change_this_bcrypt_hash

# Email (SMTP for booking confirmations)
SMTP_HOST=smtp.example.com
SMTP_PORT=587
SMTP_USER=noreply@facility.example.com.au
SMTP_PASS=smtp_password_here
EMAIL_FROM="Special Need Vehicle Rental <noreply@facility.example.com.au>"

# Security
SESSION_SECRET=generate_a_random_64_char_secret_here
CORS_ORIGIN=https://www.facilitydomain.com.au

# Locale
TZ=UTC
DISPLAY_TZ=Australia/Sydney
CURRENCY=AUD
GST_RATE=0.10

# Rate limits
RATE_LIMIT_WINDOW_MS=900000
RATE_LIMIT_MAX=100
```

9.4 Install & Run

```
# Install backend dependencies
npm install

# Create SQLite database and run migrations
npm run migrate

# Seed 100 sample vehicles (accessibility fleet)
npm run seed

# Start development server (with hot reload)
npm run dev

# Build frontend
```

```
cd frontend && npm install && npm run build && cd ..

# Start production server
npm start
```

9.5 Development URL

```
http://localhost:8080          → Public booking site
http://localhost:8080/admin    → Admin UI (use ADMIN_USERNAME / ADMIN_PASSWORD)
http://localhost:8080/api/v1   → REST API base
```

10. Database Migrations & Seeding

10.1 Migration Runner

```
// scripts/migrate.js
const Database = require('better-sqlite3');
const fs = require('fs');
const path = require('path');

const db = new Database(process.env.DATABASE_PATH || './data/rental.sqlite');
const sql = fs.readFileSync(
  path.join(__dirname, '../migrations/001_initial_schema.sql'), 'utf8'
);
db.exec(sql);
console.log('Migration complete.');
```

10.2 Seed Script (100 Sample Vehicles)

```
// scripts/seed.js – generates 100 representative vehicles
const vehicles = [
  // Wheelchair-accessible vans
  { name: 'Toyota HiAce Wheelchair Van 1', type: 'van', capacity: 4,
    accessibility_notes: 'Rear ramp, 2 wheelchair anchor points, hand controls',
    hourly_rate_cents: 2500, daily_rate_cents: 14000 },
  // ... (script generates the full 100 using a loop with varied types)
];
```

Run: `node scripts/seed.js`

11. Testing Strategy

11.1 Unit Tests (Jest)

Focus areas:

- Availability overlap algorithm (edge cases: adjacent bookings, buffer times, UTC boundary conditions)
- Price calculation with GST
- Input validation schemas (Zod)

```
npm test
```

11.2 Integration Tests

Focus areas:

- Complete booking flow: browse → select → reserve → payment form submit → confirm
- Concurrent reservation race condition (two requests for same slot simultaneously)
- Admin login, vehicle CRUD, reservation status updates
- Cancellation within / outside policy window

```
npm run test:integration
```

11.3 Accessibility Testing

- Run **axe-core** automated checks on all pages (integrate into Playwright/Cypress E2E)
- Manual keyboard-only navigation walkthrough of the booking flow
- Test with screen reader (NVDA/VoiceOver)

12. Deployment Guide

12.1 Provision VPS (Ubuntu 22.04, Sydney Region)

```
# Update system
sudo apt update && sudo apt upgrade -y

# Install Node.js 20
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs

# Install Nginx, Certbot, Git, SQLite
sudo apt install -y nginx certbot python3-certbot-nginx git sqlite3

# Install PM2
sudo npm install -g pm2
```

12.2 Nginx Configuration

```
# /etc/nginx/sites-available/rental
server {
    listen 80;
    server_name rentals.facilitydomain.com.au;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name rentals.facilitydomain.com.au;

    ssl_certificate
/etc/letsencrypt/live/rentals.facilitydomain.com.au/fullchain.pem;
    ssl_certificate_key
/etc/letsencrypt/live/rentals.facilitydomain.com.au/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;

    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains"
always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "DENY" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;

    # Serve React static build
    root /var/www/rental/frontend/dist;
    index index.html;

    location /api/ {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location / {
        try_files $uri $uri/ /index.html;
    }

    # Deny access to data directory
    location /data/ { deny all; }
}
```

12.3 PM2 Process Configuration

```
// ecosystem.config.js
module.exports = {
  apps: [{
    name: 'rental-api',
    script: './src/server.js',
```



```
env_production: {
  NODE_ENV: 'production',
  PORT: 8080
},
max_restarts: 10,
restart_delay: 5000,
error_file: '/var/log/rental/error.log',
out_file:   '/var/log/rental/out.log'
}]
};
```

```
pm2 start ecosystem.config.js --env production
pm2 save
pm2 startup    # generates systemd unit for auto-start on reboot
```

12.4 Let's Encrypt Certificate

```
sudo certbot --nginx -d rentals.facilitydomain.com.au
# Certbot auto-renews via systemd timer; verify with:
sudo systemctl status certbot.timer
```

12.5 Optional GitHub Actions CI/CD

```
# .github/workflows/deploy.yml
on:
  push:
    branches: [main]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Deploy via SSH
        uses: appleboy/ssh-action@v1
        with:
          host: ${ secrets.VPS_HOST }
          username: deploy
          key: ${ secrets.VPS_KEY }
          script: |
            cd /var/www/rental
            git pull
            npm ci --production
            cd frontend && npm ci && npm run build && cd ..
            pm2 reload rental-api
```

13. Cloud Hosting Cost Estimate

Target: $\leq$ **AUD \$35 / month**

Option A — DigitalOcean Droplet (Recommended)

Item	Monthly Cost (AUD est.)
Basic Droplet — 1 vCPU, 1 GB RAM, 25 GB SSD (Sydney region)	~\$10
Weekly automated backups (DigitalOcean)	~\$2
Domain renewal (com.au)	~\$2
Total	~\$14 / month

Option B — AWS Lightsail (Sydney ap-southeast-2)

Item	Monthly Cost (AUD est.)
Lightsail instance — 1 vCPU, 1 GB RAM, 40 GB SSD	~\$10
Manual snapshot (weekly)	~\$3
Domain (Route 53)	~\$2
Total	~\$15 / month

Option C — AWS EC2 t4g.nano (Reserved 1-year)

Item	Monthly Cost (AUD est.)
EC2 t4g.nano (1 vCPU, 512 MB RAM)	~\$4
EBS gp3 20 GB	~\$3
S3 backup (< 1 GB)	~\$0.03
Elastic IP	~\$0 (attached)
Total	~\$7–10 / month

All options are comfortably under AUD \$35/month. DigitalOcean is recommended for simplicity (managed backups, intuitive interface, Sydney availability).

14. Monitoring & Observability

14.1 Application Health

```
# Health check endpoint
GET /api/v1/health
# Returns: { "status": "ok", "db": "ok", "uptime_seconds": 12345 }
```

Configure an uptime monitor (UptimeRobot free tier, or BetterUptime) to ping `/api/v1/health` every 5 minutes and alert via email.

14.2 Log Management

- PM2 rotates logs daily (configure `pm2-logrotate` module)
- Application logs use `pino` structured JSON: timestamp, level, request ID, route, latency
- Access logs via Nginx: `/var/log/nginx/rental_access.log`
- Error alerts: pipe `pino` ERROR-level logs to an email webhook

14.3 Database Integrity

Weekly integrity check via cron:

```
# crontab -e (root or deploy user)
0 2 * * 0 sqlite3 /var/www/rental/data/rental.sqlite "PRAGMA integrity_check;" \
| mail -s "Rental DB integrity check" admin@facility.example.com.au
```

15. Backup & Recovery

15.1 Automated Daily Backup

```
#!/bin/bash
# /usr/local/bin/backup-rental-db.sh
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
DB_PATH="/var/www/rental/data/rental.sqlite"
BACKUP_DIR="/var/backups/rental"

mkdir -p "$BACKUP_DIR"
sqlite3 "$DB_PATH" ".backup '$BACKUP_DIR/rental_${TIMESTAMP}.sqlite'"

# Keep last 14 days
find "$BACKUP_DIR" -name "rental_*.sqlite" -mtime +14 -delete

# Optional: sync to object storage
# rclone copy "$BACKUP_DIR" remote:rental-backups/
```

```
# crontab entry (run at 2:30 AM Sydney time = 16:30 UTC)
30 16 * * * /usr/local/bin/backup-rental-db.sh
```

15.2 Recovery Procedure

```
# Stop application
pm2 stop rental-api
```

```
# Restore from backup
cp /var/backups/rental/rental_20260615_163001.sqlite \
  /var/www/rental/data/rental.sqlite

# Verify integrity
sqlite3 /var/www/rental/data/rental.sqlite "PRAGMA integrity_check;"

# Restart application
pm2 start rental-api
```

16. Handoff Deliverables

The following artefacts must be delivered at project handoff:

Artefact	Description
README.md	Quick-start, env vars, run commands, deployment summary
migrations/001_initial_schema.sql	Full SQL schema as documented in §4.2
scripts/seed.js	Populates 100 sample vehicles including accessibility metadata
scripts/migrate.js	Migration runner
openapi.yaml	OpenAPI 3.1 spec for all API endpoints
postman_collection.json	Postman collection with pre-configured environment
ecosystem.config.js	PM2 process configuration
nginx.conf.example	Nginx site configuration template
.env.example	All env vars with comments; no secrets
tests/	Unit and integration test suites
DEPLOYMENT.md	Step-by-step production deployment checklist
PRIVACY_POLICY.md	Draft privacy policy compliant with APPs
TERMS_CONDITIONS.md	Draft rental terms and conditions (NSW-specific)

This document is a living guide. Update version and date fields whenever material changes are made.